Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

Отчёт по лабораторной работе №12

Программирование в командном процессоре ОС UNIX. Командные файлы

Студент: ТУЙИШИМЕ Тьерри

Группа: НКАбд-05-25

Содержание

1 Цель работы 3
2 Задание 3
3 Выполнение лабораторной работы 3
4 Выводы 7
5 Ответы на контрольные вопросы 7
Список литературы 12

1 Цель работы

Научиться писать небольшие командные файлы.

2 Задание

1. Написать скрипт, который при запуске будет делать резервную копию самого себя в другую директорию
2. Написать пример командного файла, обрабатывающего любое произвольное число аргументов командной строки
3. Написать командный файл — аналог команды ls
4. Написать командный файл, который получает в качестве аргумента командной строки формат файла и вычисляет количество таких файлов в указанной директории.

3 Выполнение лабораторной работы

Я создала файл project.sh. В этом файле я написала скрипт ,который при запуске будет делать резервную копию самого себя (то есть файла, в котором содержится его исходный код) в другую директорию backup в домашнем каталоге. Затем я создала исполняемый файл и запускала его:

```
Micaca@Micaca:~$ touch project.sh
Micaca@Micaca:~$ gedit project.sh
```

Рис. 1: Создание project.sh

```
tar -cvf ~/backup/project.tar project.sh
```

Рис. 2: Скрипт

```
Micaca@Micaca:~$ chmod +x project.sh
Micaca@Micaca:~$
```

Рис. 3: Создание исполняемого файла

```
Micaca@Micaca:~$ mkdir backup
Micaca@Micaca:~$ ./project.sh
project.sh
Micaca@Micaca:~$ ls backup/
project.tar
Micaca@Micaca:~$
```

Рис. 4: Запуск файла

Я проверила с помощью cat,что копия содержит скрипт из оригинала:

```
Micaca@Micaca:~$ cat ~/backup/project.tar
project.sh0000755000175200017520000000000511517511020001216 7 0ustar
 MicacaMicacatar -cvf ~/backup/project.tar project.sh
Micaca@Micaca:~$
```

Рис. 5: Проверка копии файла

Создала другой файл project1.sh. Написала код, обрабатывающий любое произвольное число аргументов командной строки. Сделала его исполняемый и запускала его:

```
Micaca@Micaca:~$ touch project1.sh
Micaca@Micaca:~$ gedit project1.sh
Micaca@Micaca:~$ chmod +x project1.sh
```

Рис. 6: Создание project1.sh

Рис. 7: Код project1.sh

```
for A in $*
    do echo $A
done
```

```
Micaca@Micaca:~$ ./project1.sh 10 20 30 40 50 100 winner winner fidget spinner
10
20
30
40
50
100
winner
winner
fidget
spinner
Micaca@Micaca:~$
```

Рис. 8: Запуск project1.sh

Создала файл project2.sh и в нем написала программу - аналог команды ls. Она выводит информацию о возможностях доступа к файлам этого каталога и информацию о нужном каталоге:

```
Micaca@Micaca:~$ gedit project2.sh
Micaca@Micaca:~$ chmod +x project2.sh
Micaca@Micaca:~$
```

Рис. 9: Создание project2.sh

```
for A in *
do
    if test -d "$A"
    then
        echo "$A^ is a directory"
    else
        echo "$A: is a file and"
        if test -w $A
        then
            echo " is writable"
                if test -r $A
                then
                    echo "is readable"
            else
                echo "Neither raedable nor writable"
                fi
            fi
        fi
done
```

```
Open    *project2.sh    Save
1 for A in *
2 do
3     if test -d "$A"
4     then
5         echo "$A is a directory"
6     else
7         echo "$A is a file and"
8         if test -w $A
9         then
10            echo " is writable"
11            if test -r $A
12            then
13                echo "is readable"
14            else
15                echo "Neither readable nor writable"
16            fi
17        fi
```

Рис. 10: Программа project2.sh

```
Micaca@Micaca:~$ ./project2.sh
abc1 is a file and
 is writable
is readable
australia is a directory
backup is a directory
bin is a directory
conf.txt is a file and
 is writable
is readable
Desktop is a directory
Documents is a directory
Downloads is a directory
file.txt is a file and
 is writable
is readable
fun is a directory
git-extended is a directory
gitflow-avh is a directory
helloworld.cpp is a file and
 is writable
is readable
```

Рис. 11: Запуск программы

Создала файл project3.sh и в нем написала код, который олучает в качестве аргумента командной строки формат файла (.txt, .doc, .jpg, .pdf и т.д.) и вычисляет количество таких файлов в указанной директории.

```
echo "Enter file format: "
read format
echo "Enter directory: "
read directory
find "${directory}" -name ".${format}" -type f | wc -l
```

Рис. 12: Код project3.sh

```
echo "Enter file format: "
read format
echo "Enter directory: "
read directory
find "${directory}" -name ".${format}" -type f | wc -l
```

```
Micaca@Micaca:~$ gedit project3.sh
Micaca@Micaca:~$ chmod +x project3.sh
Micaca@Micaca:~$ ./project3.sh
Enter file format:
jpg
Enter directory:
/home/Micaca
75
Micaca@Micaca:~$
```

Рис. 13: Запуск project3.sh

4 Выводы

При выполнении данной работы я научилась писать небольшие командные файлы.

5 Ответы на контрольные вопросы

1. Командные процессоры или оболочки - это программы, позволяющие пользователю взаимодействовать с компьютером. Их можно рассматривать как настоящие интерпретируемые языки, которые воспринимают команды пользователя и обрабатывают их. Поэтому командные процессоры также называют интерпретаторами команд. На языках оболочек можно писать программы и выполнять их подобно любым другим программам. UNIX обладает большим количеством оболочек. Наиболее популярными являются следующие четыре оболочки: –оболочка Борна (Bourne) - первоначальная командная оболочка UNIX: базовый, но полный набор функций; –C-оболочка - добавка университета Беркли к коллекции оболочек: она надстраивается над оболочкой Борна, используя C-подобный синтаксис команд, и сохраняет историю выполненных команд; –оболочка Корна - напоминает оболочку C, но операторы управления программой совместимы с операторами оболочки Борна; –BASH - сокращение от Bourne Again Shell (опять оболочка Борна), в основе своей совмещает свойства оболочек C и Корна (разработка компании Free Software Foundation).

2. POSIX (Portable Operating System Interface for Computer Environments)- интерфейс переносимой операционной системы для компьютерных сред. Представляет собой набор стандартов, подготовленных институтом инженеров по электронике и радиотехники (IEEE), который определяет различные аспекты построения операционной системы. POSIX включает такие темы, как программный интерфейс, безопасность, работа с сетями и графический интерфейс. POSIX-совместимые оболочки являются будущим поколением оболочек UNIX и других ОС. Windows NT рекламируется как система, удовлетворяющая POSIX-стандартам. POSIX-совместимые оболочки разработаны на базе оболочки Корна; фонд бесплатного программного обеспечения (Free Software Foundation) работает над тем, чтобы и оболочку BASH сделать POSIX-совместимой.

3. Командный процессор bash обеспечивает возможность использования переменных типа строка символов. Имена переменных могут быть выбраны пользователем. Пользователь имеет возможность присвоить переменной значение некоторой строки символов. Например, команда mark=/usr/andy/bin присваивает значение строки символов /usr/andy/bin переменной mark типа строка символов. Значение, присвоенное некоторой переменной, может быть впоследствии использовано. Для этого в соответствующем месте командной строки должно быть употреблено имя этой переменной, которому предшествует метасимвол Напримеркоманда {имя переменной} например, использование команд b=/tmp/andy-ls -l myfile > blsприведеткпереназначениюстандартноговыводакомандыlsстерминаланафайл/ $tmp/andy - ls$, аиспользованиекоманды$ls - l$ >bls приведет к подстановке в командную строку значения переменной bls. Если переменной bls не было предварительно присвоено никакого значения, то ее значением является символ пробел. Оболочка bash позволяет создание массивов. Для создания массива используется команда set с флагом -A. За флагом следует имя переменной, а затем список значений, разделенных пробелом. Например,set -A states Delaware Michigan “New Jersey” Далее можно сделать добавление в массив, например, states[49]=Alaska. Индексация массивов начинается с нулевого элемента.

4. Команда let является показателем того, что последующие аргументы представляют собой выражение, подлежащее вычислению. Простейшее выражение - это единичный терм (term), обычно целочисленный. Целые числа можно записывать как последовательность цифр или в любом базовом формате. Этот формат — radix#number, где radix (основание системы счисления) - любое число не более 26. Для большинства команд основания систем счисления это - 2 (двоичная), 8 (восьмеричная) и 16 (шестнадцатеричная). Простейшими математическими выражениями являются сложение (+), вычитание (-), умножение (*), целочисленное деление (/) и целочисленный остаток (%). Команда let берет два операнда и присваивает их переменной.

5. Оператор Синтаксис Результат ! !exp Если exp равно 0, возвращает 1; иначе 0 != exp1 !=exp2 Если exp1 не равно exp2, возвращает 1; иначе 0 % exp1%exp2 Возвращает остаток от деления exp1 на exp2 %= var=%exp Присваивает остаток от деления var на exp переменной var & exp1&exp2 Возвращает побитовое AND выражений exp1 и exp2 && exp1&&exp2 Если и exp1 и exp2 не равны нулю, возвращает 1; иначе 0 &= var &= exp Присваивает var побитовое AND перемен- ных var и выражения exp * exp1 * exp2 Умножает exp1 на exp2 = *var* = exp Умножает exp на значение var и присваивает результат переменной var + exp1 + exp2 Складывает exp1 и exp2 += var += exp Складывает exp со значением var и результат присваивает var - -exp Операция отрицания exp (называется унарный минус) - expl - exp2 Вычитает exp2 из exp1 -= var -= exp Вычитает exp из значения var и присваи- вает результат var / exp / exp2 Делит exp1 на exp2 /= var /= exp Делит var на exp и присваивает результат var < expl < exp2

Если exp1 меньше, чем exp2, возвращает 1, иначе возвращает 0 « exp1« exp2 Сдвигает exp1 влево на exp2 бит «= var «= exp Побитовый сдвиг влево значения var на exp <= expl <= exp2 Если exp1 меньше, или равно exp2, возвра- щает 1; иначе возвращает 0 = var = exp Присваивает значение exp переменной va == exp1==exp2 Если exp1 равно exp2.

Возвращает 1; иначе возвращает 0 > exp1 > exp2 1 если exp1 больше, чем exp2; иначе 0 >= exp1 >= exp2 1 если exp1 больше, или равно exp2; иначе 0 » exp » exp2 Сдвигает exp1 вправо на exp2 бит »= var »=exp Побитовый сдвиг вправо значения var на exp ^ exp1 ^ exp2 Исключающее OR

выражений exp1 и exp2 ^= var ^= exp Присваивает var побитовое исключающее OR var и exp | exp1 | exp2 Побитовое OR выражений exp1 и exp2 |= var |= exp Присваивает var «исключающее OR» пе- ременой var и выражения exp || exp1 || exp2 1 если или exp1 или exp2 являются нену- левыми значениями; иначе 0 ~ ~exp Побитовое дополнение до exp.

6. Что означает операция (( ))? Условия оболочки bash.

7. Имя переменной (идентификатор) — это строка символов, которая отличает эту переменную от других объектов программы (идентифицирует переменную в программе). При задании имен переменным нужно соблюдать следующие правила: § первым символом имени должна быть буква. Остальные символы — буквы и цифры (прописные и строчные буквы различаются). Можно использовать символ «_»; § в имени нельзя использовать символ «.»; § число символов в имени не должно превышать 255; § имя переменной не должно совпадать с зарезервированными (служебными) словами языка. Var1, PATH, trash, mon, day, PS1, PS2 Другие стандартные переменные: –HOME — имя домашнего каталога пользователя. Если команда cd вводится без аргументов, то происходит переход в каталог, указан- ный в этой переменной . –IFS — последовательность символов, являющихся разделителями в командной строке. Это символы пробел, табуляция и перевод строки(new line). –MAIL — командный процессор каждый раз перед выводом на экран промптера проверяет содержимое файла, имя которого указано в этой переменной, и если содержимое этого файла изменилось с момента последнего ввода из него, то перед тем как вывести на терминал промптер, командный процессор выводит на терминал сообщение You have mail (у Вас есть почта). –TERM — тип используемого терминала. –LOGNAME — содержит регистрационное имя пользователя, которое устанавливается автоматически при входе в систему. В командном процессоре Си имеется еще несколько стандартных переменных. Значение всех переменных можно просмотреть с помощью команды set.

8. Такие символы, как ’ < > * ? | ” & являются метасимволами и имеют для командного процессора специальный смысл.

9. Снятие специального смысла с метасимвола называется экранированием метасимвола. Экранирование может быть осуществлено с помощью предшествующего метасимволу символа, который, в свою очередь, является метасимволом. Для экранирования группы метасимволов, ее нужно заключить в одинарные кавычки. Строка, заключенная в двойные кавычки, экранирует все метасимволы, кроме $, ’ , , “. Например,–echo выведет на экран символ,–echo ab’|’cdвыдаст строку ab/cd.

10. Последовательность команд может быть помещена в текстовый файл. Такой файл называется командным. Далее этот файл можно выполнить по команде bash командный_файл [аргументы] Чтобы не вводить каждый раз

последовательности символов bash, необходимо изменить код защиты этого командного файла, обеспечив доступ к этому файлу по выполнению. Это может быть сделано с помощью команды chmod +x имя_файла Теперь можно вызывать свой командный файл на выполнение просто, вводя его имя с терминала так, как будто он является выполняемой программой. Командный процессор распознает, что в Вашем файле на самом деле хранится не выполняемая программа, а программа, написанная на языке программирования оболочки, и осуществит ее интерпретацию.

11. Группу команд можно объединить в функцию. Для этого существует ключевое слово function, после которого следует имя функции и список команд, заключенных в фигурные скобки. Удалить функцию можно с помощью команды unset с флагом-f. Команда typeset имеет четыре опции для работы с функциями: -f — перечисляет определенные на текущий момент функции; –-ft— при последующем вызове функции инициирует ее трассировку; –-fx— экспортирует все перечисленные функции в любые дочерние программы оболочек; –-

fu— обозначает указанные функции как автоматически загружаемые. Автоматически загружаемые функции хранятся в командных файлах, а при их вызове оболочка просматривает переменную FPATH, отыскивая файл с одноименными именами функций, загружает его и вызывает эти функции.

12. ls -lrt Если есть d, то является файл каталогом

13. Используется команда set с флагом -A. За флагом следует имя переменной, а затем список значений, разделенных пробелом. Например, set -A states Delaware Michigan “New Jersey” Далее можно сделать добавление в массив, например, states[49]=Alaska . Индексация массивов начинается с нулевого элемента. В командном процессоре Си имеется еще несколько стандартных переменных. Значение всех переменных можно просмотреть с помощью команды set. Наиболее распространенным является сокращение, избавляющееся от слова let в программах оболочек. Если объявить переменные целыми значениями, любое присвоение автоматически трактуется как арифметическое. Используйте typeset -i для объявления и присвоения переменной, и при последующем использовании она становится целой. Или можете использовать ключевое слово integer (псевдоним для typeset -l) и объявлять переменные целыми. Таким образом, выражения типа x=y+z воспринимаются как арифметические. Группу команд можно объединить в функцию. Для этого существует ключевое слово function , после которого следует имя функции и список команд, заключенных в фигурные скобки. Удалить функцию можно с помощью команды unset с флагом -f . Команда typeset имеет четыре опции для работы с функциями: – -f — перечисляет определенные на текущий момент функции; – -ft — при последующем вызове функции инициирует ее трассировку; – -fx — экспортирует все перечисленные функции в любые дочерние программы оболочек; – -fu — обозначает указанные функции как автоматически загружаемые. Автоматически загружаемые функции хранятся в командных файлах, а при их вызове оболочка просматривает переменную FPATH , отыскивая файл с одноименными именами функций, загружает его и вызывает эти функции. В переменные mon и day будут считаны соответствующие значения, введенные с клавиатуры, а переменная trash нужна

для того, чтобы отобрать всю избыточно введенную информацию и игнорировать ее. Изъять переменную из программы можно с помощью команды unset.

14. Символ $ является метасимволом командного процессора. Он используется, в частности, для ссылки на параметры, точнее, для получения их значений в командном файле. В командный файл можно передать до девяти параметров. При использовании где-либо в командном файле комбинации символов $i, где 0 < i < 10, вместо нее будет осуществлена подстановка значения параметра с порядковым номером i, т.е. аргумента командного файла с порядковым номером i. Использование комбинации символов $0 приводит к подстановке вместо нее имени данного командного файла. Примере: пусть к командному файлу where имеется доступ по выполнению и этот командный файл содержит следующий конвейер: who | grep $1 Если Вы введете с терминала команду: where andy, то в случае, если пользователь, зарегистрированный в ОС UNIX под именем andy, в данный момент работает в ОС UNIX, на терминал будет выведена строка, содержащая номер терминала, используемого указанным пользователем. Если же в данный момент этот пользователь не работает в ОС UNIX, то на терминал не будет выведено ничего. Команда grep производит контекстный поиск в тексте, поступающем со стандартного ввода, для нахождения в этом тексте строк, содержащих последовательности символов, переданные ей в качестве аргументов, и выводит результаты своей работы на стандартный вывод. В этом примере команда grep используется как фильтр, обеспечивающий ввод со стандартного ввода и вывод всех строк, содержащих последовательность символов andy, на стандартный вывод. В ходе интерпретации этого файла командным процессором вместо комбинации символов $1 осуществляется подстановка значения первого и единственного параметра andy. Если предположить, что пользователь, зарегистрированный в ОС UNIX под именем andy, в данный момент работает в

ОС UNIX, то на терминале Вы увидите примерно следующее: $ where andy andy ttyG Jan 14 09:12 $ Определим функцию, которая изменяет каталог и печатает список файлов: $ function clist { > cd $1 > ls > }. Теперь при вызове команды clist каталог будет изменен каталог и выведено его содержимое.

15. $* — отображается вся командная строка или параметры оболочки;

$? — код завершения последней выполненной команды;

$$ — уникальный идентификатор процесса, в рамках которого выполняется командный процессор;

$! — номер процесса, в рамках которого выполняется последняя вызванная на выполнение в командном режиме команда;

$- — значение флагов командного процессора;

You can’t use ‘macro parameter character #’ in math mode ${#} — возвращает целое число — количество слов, которые были результатом $;

${#name} — возвращает целое значение длины строки в переменной name;

${name[n]} — обращение к n-ному элементу массива;

${name[*]} — перечисляет все элементы массива, разделенные пробелом;

${name[@]} — то же самое, но позволяет учитывать символы пробелы в самих переменных;

${name:-value} — если значение переменной name не определено, то оно будет заменено на указанное value;

${name:value} — проверяется факт существования переменной;

${name=value} — если name не определено, то ему присваивается значение value;

${name?value} — останавливает выполнение, если имя переменной не определено, и выводит value, как сообщение об ошибке;

этовыражениеработаетпротивоположно {name-value}. Если переменная определена, то подставляется value;

${name#pattern} — представляет значение переменной name с удаленным самым коротким левым образцом (pattern);

${#name[*]} и ${#name[@]} — эти выражения возвращают количество элементов в массиве name.

$# вместо нее будет осуществлена подстановка числа параметров, указанных в командной строке при вызове данного командного файла на выпол

Список литературы

Архитектура ЭВМ